

Université Privée de Fès

Département Informatique



Centre Hospitalier Universitaire Hassan II de Fès

Rapport de Stage de Fin d'Année

PICU

Plateforme Intelligente de Coordination des Urgences

Présenté par :

Younes Basir

Année Universitaire :

2024-2025

Remerciements

Je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce stage et à l'élaboration de ce rapport.

Je remercie tout d'abord mon encadrant de stage au sein du Centre Hospitalier Universitaire Hassan II de Fès, pour sa disponibilité, ses conseils précieux et son accompagnement tout au long de cette période. Sa confiance et son expertise m'ont permis de mener à bien ce projet dans les meilleures conditions.

Je tiens également à remercier l'ensemble du personnel du département informatique du CHU Hassan II pour leur accueil chaleureux et leur collaboration. Leur professionnalisme et leur volonté de partager leurs connaissances ont grandement enrichi mon expérience.

Mes remerciements vont également à mes professeurs de l'Université Privée de Fès, notamment ceux du département informatique, pour la qualité de l'enseignement reçu et la préparation rigoureuse qui m'a permis d'acquérir les compétences nécessaires à la réalisation de ce projet.

Je remercie également l'ensemble de l'équipe pédagogique de l'UPF pour leur suivi et leur soutien tout au long de mon parcours universitaire.

Enfin, je remercie ma famille pour son soutien inconditionnel, sa patience et son encouragement constants tout au long de mes études et particulièrement durant la réalisation de ce stage.

Résumé

Ce rapport de stage présente la conception, la réalisation et l'évaluation de PICU (Plateforme Intelligente de Coordination des Urgences), une application web destinée au service des urgences du Centre Hospitalier Universitaire Hassan II de Fès. Le projet répond à un double constat : la dispersion de l'information entre les acteurs (accueil, triage, médecine) et l'absence de visibilité en temps réel sur l'occupation des zones, des salles et des lits.

Plutôt que de décrire uniquement des fonctionnalités, ce rapport met l'accent sur la **démarche d'ingénierie logicielle** : analyse des besoins, méthodologie de développement, modélisation, conception de la base de données, architecture, assurance qualité, sécurité et industrialisation. L'originalité du travail réside dans la transformation d'un processus hospitalier informel en un **workflow transactionnel** explicite : une machine à états de la visite (`waiting`, `placed`, `in_treatment`, puis états terminaux), des transitions autorisées, des priorités cliniques P1–P5, des files d'attente calculées et une gestion des ressources verrouillée en base de données.

La plateforme est construite sur une architecture client-serveur : un frontend React 19 / Vite / TailwindCSS, un backend Node.js / Express exposant une API REST, une base PostgreSQL (hébergée sur Neon) et une communication temps réel par **Socket.IO**. La sécurité repose sur un contrôle d'accès par rôle et capacités (RBAC), des jetons JWT, des transactions PostgreSQL et une piste d'audit exhaustive (`visit_events`). Des modules avancés complètent le socle : supervision opérationnelle avec indicateurs SLA, gestion du cycle de vie des lits, demandes de transfert et de relève, examens et tâches de soin, suivi analytique, ainsi qu'une couche d'**assistance IA** fondée sur NVIDIA NIM (triage, flux de lits, capacité, documentation) fonctionnant en mode supervisé avec approbation humaine.

La qualité est vérifiée par une stratégie de tests explicite : tests unitaires et d'intégrité basés sur `node:test` (invariants de ressources, règles de supervision, intelligence d'enregistrement, analytics), tests fonctionnels par rôle, tests de concurrence sur l'affectation des lits et scénario de bout en bout. L'industrialisation s'appuie sur Git, des hooks pre-commit et un pipeline GitHub Actions « secret guard ».

Ce rapport est organisé en six chapitres. Le premier présente l'organisme d'accueil et

le contexte du projet. Le deuxième formalise l'analyse et la spécification des besoins. Le troisième expose la méthodologie et la conception. Le quatrième détaille la réalisation. Le cinquième traite de l'assurance qualité, de la sécurité et de la CI/CD. Le sixième dresse l'évaluation, le bilan et les perspectives, notamment l'assistance IA NVIDIA NIM.

Mots-clés : PICU, Urgences, Workflow transactionnel, Machine à états, Triage IA, RBAC, PostgreSQL, Socket.IO, CI/CD, Qualité logicielle

Abstract

This internship report presents the design, implementation and evaluation of PICU (Intelligent Emergency Coordination Platform), a web application for the emergency department of the Hassan II University Hospital in Fez. The project addresses two problems : information dispersion between actors (registration, triage, physicians) and the lack of real-time visibility over zone, room and bed occupancy.

Rather than describing features only, this report emphasizes the **software engineering process** : requirements analysis, development methodology, modelling, database design, architecture, quality assurance, security and deployment automation. The core contribution is the transformation of an informal hospital process into an explicit **transactional workflow** : a visit state machine (`waiting`, `placed`, `in_treatment`, then terminal states), allowed transitions, P1–P5 clinical priorities, computed waiting lists and database-locked resource management.

The platform uses a client-server architecture : a React 19 / Vite / TailwindCSS frontend, a Node.js / Express REST API, a PostgreSQL database (hosted on Neon) and **Socket.IO** real-time communication. Security relies on role and capability-based access control (RBAC), JWT tokens, PostgreSQL transactions and a complete audit trail (`visit_events`). Advanced modules complete the foundation : supervision with SLA indicators, bed lifecycle management, transfer and handoff requests, exams and care tasks, analytics, and an **AI assistance** layer built on NVIDIA NIM (triage, bed flow, capacity, documentation) operating under human supervision.

Quality is verified through an explicit testing strategy : unit and integrity tests based on `node:test`, role-based functional tests, concurrency tests on bed assignment, and an end-to-end scenario. Delivery automation relies on Git, pre-commit hooks and a GitHub Actions "secret guard" pipeline.

The report is organised into six chapters : host organization and context ; requirements analysis ; methodology and design ; implementation ; quality assurance, security and CI/CD ; and evaluation, review and perspectives.

Keywords : PICU, Emergency Department, Transactional Workflow, State Machine, AI Triage, RBAC, PostgreSQL, Socket.IO, CI/CD, Software Quality

Table des matières

Remerciements 1

Résumé 2

Abstract 4

Table des matières 5

Table des figures 9

Liste des tableaux 10

Liste des acronymes 11

Introduction générale 12

0.1 Contexte général . . . 12

0.2 Problématique . . . 12

0.3 Objectifs du projet . . . 13

0.4 Démarche d'ingénierie adoptée . . . 13

0.5 Méthodologie et outils . . . 13

0.6 Organisation du rapport . . . 14

1 Organisme d'accueil et contexte du projet 15

1.1 Introduction . . . 15

1.2 Présentation de l'organisme d'accueil . . . 15

1.3 Cadre du stage . . . 16

1.4 Fonctionnement actuel du service des urgences . . . 16

1.5 Limites du fonctionnement actuel . . . 17

1.6 Problématique et solution proposée . . . 17

1.7 Distinction entre prototype et système validé . . . 18

1.8 Conclusion 18

2 Analyse et spécification des besoins 19

2.1 Introduction 19
2.2 Étude du processus des urgences 19
2.3 Identification des acteurs 20
2.4 Besoins fonctionnels 20
2.5 Besoins non fonctionnels 22
2.6 Règles métier structurantes 22
2.7 Diagramme de cas d'utilisation 23
2.8 Description des principaux cas d'utilisation 23
2.9 Matrice de traçabilité des exigences 25
2.10 Gestion des risques 25
2.11 Conclusion 25

3 Méthodologie et conception du système 26

3.1 Introduction 26
3.2 Méthodologie de développement 26
3.3 Architecture logicielle 27
3.4 Conception du workflow : machine à états 28
3.5 Diagrammes UML 30
3.6 Conception de la base de données 32
3.7 Choix technologiques et justification 34
3.8 Conclusion 34

4 Réalisation de la plateforme PICU 35

4.1 Introduction 35
4.2 Environnement de développement 35
4.3 Module d'authentification et de gestion des accès 35
4.4 Module patients et visites 36
4.5 Module file d'attente et appel 36
4.6 Module de triage 37
4.7 Module de gestion des ressources 37

4.8 Module de prise en charge et consultations 38
4.9 Module transferts, relèves, examens et tâches 38
4.10 Module supervision et alertes 39
4.11 Tableaux de bord par rôle 40
4.12 Synchronisation temps réel 41
4.13 Couche d'assistance IA 41
4.14 Conclusion 42

5 Assurance qualité, sécurité et CI/CD 43
5.1 Introduction 43
5.2 Stratégie de test 43
5.3 Tests automatisés et résultats 44
5.4 Cas d'étude : concurrence sur l'affectation d'un lit 45
5.5 Tests fonctionnels et bout en bout 45
5.6 Gestion des erreurs 45
5.7 Sécurité et intégrité 45
5.8 Qualité du code 46
5.9 Industrialisation et CI/CD 46
5.10 Conclusion 47

6 Évaluation, bilan et perspectives 48
6.1 Introduction 48
6.2 Évaluation du projet 48
6.3 Difficultés rencontrées et solutions apportées 50
6.4 Limites de la plateforme 50
6.5 Compétences acquises 50
6.6 Perspectives 51
6.7 Conclusion 51

Conclusion générale 52

Bibliographie 54

Webographie 55

Annexes **56**

6.8 Annexe A — Diagrammes UML 56

6.9 Annexe B — Modèle de base de données 56

6.10 Annexe C — Documentation des API 56

6.11 Annexe D — Captures d'écran de la plateforme 57

6.12 Annexe E — Scénarios et commandes de test 57

6.13 Annexe F — Matrice de traçabilité complète 58

Table des figures

2.1 Diagramme de cas d'utilisation de PICU 23

3.1 Architecture en trois couches et communications REST/Socket.IO 28
3.2 Diagramme de séquence — Enregistrement d'un patient 30
3.3 Diagramme d'activité — Circuit de triage et de prise en charge 31
3.4 Diagramme de classes de PICU 32

4.1 Page de connexion de PICU 36
4.2 Dashboard Infirmier — triage, plan des lits et file 38
4.3 Dashboard Médecin — patients actifs et consultations 38
4.4 Dashboard Chef de service — centre de commande 40
4.5 Dashboard Accueil — file d'attente et enregistrement 40
4.6 Dashboard Administrateur — configuration du système 41

Liste des tableaux

2.1 Acteurs du système et responsabilités 20
2.2 Exigences non fonctionnelles 22
2.3 Extrait de la matrice Exigence → Implémentation → Test 25
2.4 Risques identifiés et mesures de prévention 25

3.1 Machine à états du statut de visite 28
3.2 Actions de workflow et règles 29
3.3 Dictionnaire de données — extrait 33
3.4 Choix technologiques et justification 34

4.1 Capacités par rôle (extrait) 36
4.2 Seuils SLA par défaut (chef de service) 39
4.3 Principaux événements temps réel 41
4.4 Politique d'actions IA (extrait) 42

5.1 Matrice de tests — extrait 44

6.1 Objectifs, réalisation et validation 49
6.2 Difficultés et solutions 50
6.3 Matrice Exigence → Implémentation → Test 58

Liste des acronymes

API	Application Programming Interface
BNF	Besoin Non Fonctionnel
CHU	Centre Hospitalier Universitaire
CI/CD	Intégration Continue / Livraison Continue
CRUD	Create, Read, Update, Delete
FHIR	Fast Healthcare Interoperability Resources
HL7	Health Level Seven
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
JSONB	Type JSON binaire de PostgreSQL
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
NIM	NVIDIA Inference Microservices
PICU	Plateforme Intelligente de Coordination des Urgences
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SIH	Système d'Information Hospitalier
SLA	Service Level Agreement
UML	Unified Modeling Language

Introduction générale

0.1 Contexte général

Les services des urgences hospitaliers constituent un maillon critique du système de santé : ils assurent la prise en charge immédiate de patients dont l'état peut s'aggraver en quelques minutes. Le Centre Hospitalier Universitaire Hassan II de Fès, établissement de référence régional, accueille en permanence un flux important de patients adultes et pédiatriques. La qualité de cet accueil dépend d'une coordination rigoureuse entre l'accueil administratif, le triage clinique, l'orientation vers les zones spécialisées et la disponibilité des ressources (salles, lits, personnel).

Or, dans de nombreux services, cette coordination reste largement manuelle : transmissions orales, supports papier et outils non intégrés. Il en résulte des pertes d'information, des délais de prise en charge difficiles à mesurer et une visibilité limitée sur l'occupation réelle des zones.

0.2 Problématique

La problématique centrale de ce stage peut être formulée ainsi :

> *Comment concevoir et réaliser une plateforme numérique capable de formaliser le workflow des urgences, de garantir la cohérence et la traçabilité du parcours patient de l'arrivée à la sortie, d'offrir une vision temps réel partagée entre les rôles, tout en intégrant une assistance IA sous contrôle humain ?*

Cette problématique recouvre trois difficultés techniques : l'absence d'un modèle d'état explicite du parcours patient ; la gestion concurrente des ressources partagées (deux infirmiers ne doivent jamais affecter le même lit) ; et la définition d'une frontière stricte entre les décisions automatisables et les décisions strictement cliniques.

0.3 Objectifs du projet

Le projet PICU (Plateforme Intelligente de Coordination des Urgences) poursuit les objectifs suivants :

— **formaliser** le parcours patient en une machine à états transactionnelle, source de vérité unique ;

— **centraliser** la gestion des patients, des visites, des ressources et des équipes de salle ;

— **offrir** une vision en temps réel de l'activité, diffusée par Socket.IO à tous les postes ;

— **garantir** l'intégrité et la traçabilité de chaque décision (`visit_events`) ;

— **assurer** la sécurité par un contrôle d'accès par rôle et capacités (RBAC) et des jetons JWT ;

— **assister** les équipes par une couche IA (triage, lits, capacité, documentation) strictement supervisée ;

— **industrialiser** le cycle de vie du code (versioning, hooks, intégration continue).

0.4 Démarche d'ingénierie adoptée

Le rapport met délibérément l'accent sur la démarche d'ingénierie plutôt que sur une simple énumération de fonctionnalités. Pour chaque décision majeure, nous nous efforçons de présenter le problème initial, les solutions envisagées, la solution retenue, sa justification technique et la manière dont son bon fonctionnement a été vérifié. Le projet a été mené de façon itérative, organisé autour de jalons versionnés : socle transactionnel, enrichissement du parcours (V3), supervision et ressources (V4), puis couche d'assistance IA.

0.5 Méthodologie et outils

La méthodologie combine une approche incrémentale, une modélisation UML en amont (cas d'utilisation, états, séquence, classes), une conception de base de données normalisée et un cycle qualité continu. Les principaux outils sont :

— **Git** pour la gestion de versions, avec une convention de commits et des hooks `pre-commit` ;

— **GitHub Actions** pour les vérifications automatiques (pipeline « secret guard ») ;

— **Node.js** et le lanceur de tests natif `node:test` pour l'exécution automatisée ;

— **Draw.io / Mermaid** pour les diagrammes UML ;

— **Neon** comme hébergement PostgreSQL managé.

0.6 Organisation du rapport

Le rapport est structuré en six chapitres :

— **Chapitre 1** : Organisme d'accueil et contexte du projet ;

— **Chapitre 2** : Analyse et spécification des besoins ;

— **Chapitre 3** : Méthodologie et conception du système ;

— **Chapitre 4** : Réalisation de la plateforme PICU ;

— **Chapitre 5** : Assurance qualité, sécurité et CI/CD ;

— **Chapitre 6** : Évaluation, bilan et perspectives.

Le document se termine par une conclusion générale, une bibliographie, une webographie et des annexes.

Chapitre 1

Organisme d'accueil et contexte du projet

1.1 Introduction

Ce chapitre présente l'organisme d'accueil, le cadre du stage, le fonctionnement actuel du service des urgences, les limites identifiées et la solution proposée. Il pose ainsi le *pourquoi* du projet avant d'aborder le *comment* dans les chapitres suivants.

1.2 Présentation de l'organisme d'accueil

1.2.1 Le CHU Hassan II de Fès

Le Centre Hospitalier Universitaire Hassan II de Fès est un établissement public de santé de référence pour la région Fès–Meknès. Il assure trois missions indissociables :

— **Soins** : prise en charge des urgences vitales et non vitales, hospitalisation et spécialités ;

— **Enseignement** : formation des étudiants en médecine, des internes et du personnel paramédical ;

— **Recherche** : développement de projets innovants, dont l'informatisation des services cliniques.

1.2.2 Le service des urgences

Le service des urgences fonctionne 24 heures sur 24 et reçoit des patients adultes et pédiatriques. Il est organisé en zones spécialisées : Triage, Déchocage, Pédiatrie, Médecine,

Chirurgie et Observation. Chaque zone regroupe des salles, des lits et un personnel affecté selon la charge. Le Déchocage et la zone critique prennent en charge les urgences vitales, tandis que les zones Médecine et Chirurgie absorbent la majorité du flux.

1.2.3 Le Département Informatique

Le stage s'est déroulé au sein du Département Informatique de l'établissement, en lien fonctionnel avec l'encadrement médical du service des urgences. Ce positionnement a permis de confronter en continu les choix techniques aux contraintes cliniques et organisationnelles réelles.

1.3 Cadre du stage

1.3.1 Objet et périmètre

L'objet du stage est la conception et la réalisation d'une plateforme de coordination des urgences, PICU, couvrant le parcours du patient depuis son enregistrement à l'accueil jusqu'à la clôture de sa prise en charge. Le périmètre fonctionnel comprend l'accueil, le triage, la file d'attente, l'orientation et le placement, la prise en charge médicale, la gestion des ressources et la supervision.

1.3.2 Livrables attendus

— une application web fonctionnelle (frontend, backend, base de données) ;

— un modèle de données et une documentation d'architecture ;

— une stratégie de tests et des résultats de validation ;

— le présent rapport de stage et un support de soutenance.

1.4 Fonctionnement actuel du service des urgences

1.4.1 Accueil et enregistrement

Le réceptionniste recueille l'identité du patient, le motif de consultation et le mode d'arrivée. Un dossier administratif est créé ou mis à jour. Dans le fonctionnement manuel, cette étape repose sur des saisies répétées et des supports hétérogènes.

1.4.2 Triage clinique

Une infirmière trieuse évalue la gravité et attribue une priorité, généralement de P1 (urgent vital) à P5 (non urgent). Le triage est une décision clinique qui conditionne l'ordre de prise en charge.

1.4.3 Orientation et prise en charge

Après triage, le patient est orienté vers une zone, appelé depuis la file d'attente, placé sur une ressource (lit, fauteuil, box) puis pris en charge par un médecin. En cas d'erreur d'orientation, le médecin doit pouvoir transférer le patient vers une autre spécialité.

1.4.4 Gestion de la file d'attente et des ressources

La file est organisée par priorité clinique puis par ordre d'arrivée. La disponibilité des lits et la charge des salles déterminent la fluidité du service.

1.5 Limites du fonctionnement actuel

L'étude de l'existant met en évidence plusieurs limites :

— **absence de visibilité temps réel** sur l'occupation des zones et des lits ;

— **coordination difficile** entre accueil, soins et médecine, faute d'un référentiel commun ;

— **ressaisies et pertes d'information**, sans traçabilité fiable des décisions ;

— **risque de conflits d'affectation** lorsque plusieurs soignants manipulent les mêmes ressources ;

— **difficulté à mesurer la performance** (temps d'attente, goulots d'étranglement, qualité du triage).

Ces limites justifient une refonte numérique dont le cœur n'est pas un écran supplémentaire, mais un **modèle de processus explicite et transactionnel**.

1.6 Problématique et solution proposée

1.6.1 Problématique

Telle que formulée en introduction, la problématique interroge la capacité à formaliser le parcours patient, à garantir sa cohérence concurrente et à offrir une vision partagée en

y intégrant une assistance IA supervisée.

1.6.2 Solution proposée

PICU est une application web centralisée offrant :

— des **tableaux de bord par rôle** (accueil, infirmier, médecin, chef de service, administrateur) ;

— un **moteur de workflow transactionnel** où chaque action de visite est atomique et auditable ;

— une **gestion unifiée** des patients, des visites, des ressources et des équipes de salle ;

— une **communication temps réel** par Socket.IO synchronisant tous les postes ;

— une **couche de supervision** (SLA, alertes, blocages) pour le chef de service ;

— une **assistance IA** optionnelle et supervisée (triage, lits, capacité, documentation).

1.6.3 Apports attendus

La valeur attendue est triple : *organisationnelle* (réduction des délais et de la charge cognitive), *qualité* (traçabilité et fiabilité des données) et *décisionnelle* (indicateurs objectifs pour piloter le service).

1.7 Distinction entre prototype et système validé

Il importe de préciser le statut du livrable. PICU est une **plateforme développée et évaluée dans le cadre du stage**. Elle n'est pas déployée en production clinique et n'a pas fait l'objet d'une validation médicale officielle par l'établissement. Les fonctionnalités et les modules d'IA y sont donc présentés **selon leur état réel d'implémentation** : socle transactionnel mature, modules V3/V4 opérationnels en environnement de démonstration, et assistance IA disponible mais en mode supervisé, non autonome en production. Cette honnêteté de cadrage est une exigence d'ingénierie pour une application de santé.

1.8 Conclusion

Ce chapitre a présenté l'organisme d'accueil, le fonctionnement des urgences, ses limites et la solution PICU, en insistant sur la nécessité d'un workflow explicite et traçable. Le chapitre suivant formalise l'analyse et la spécification des besoins.

Chapitre 2

Analyse et spécification des besoins

2.1 Introduction

Ce chapitre transforme les constats du chapitre précédent en exigences exploitables. Il identifie les acteurs, distingue les besoins fonctionnels et non fonctionnels, formalise les **règles métier** structurantes, présente les cas d'utilisation puis la traçabilité entre exigences, implémentation et tests. Cette formalisation constitue le socle de la conception.

2.2 Étude du processus des urgences

2.2.1 Processus cible

Le processus cible se décompose en étapes séquentielles et vérifiables :

1. **Enregistrement** : le réceptionniste crée le patient et la visite, fixe la priorité d'enregistrement et le motif ;
2. **Triage** : l'infirmier confirme ou ajuste la priorité clinique et l'orientation ;
3. **Appel** : le patient est appelé, la file restant ordonnée par priorité puis ancienneté ;
4. **Placement** : le patient est affecté à une ressource libre et compatible ;
5. **Prise en charge** : le médecin démarre la consultation, la complète et la clôture ;
6. **Sortie ou réorientation** : la visite se termine par un outcome (sortie, hospitalisation, transfert), ou le patient est réorienté.

2.2.2 Flux d'états

Pour rendre ce processus informatisable, chaque étape correspond à une transition d'état explicite de la visite. La machine à états est détaillée au chapitre 3 ; elle est la colonne vertébrale du système.

2.3 Identification des acteurs

Table 2.1 – Acteurs du système et responsabilités

Acteur	**Responsabilités principales**
Réceptionniste	Enregistrer les patients, créer les visites, appeler, déclencher un flag critique.
Infirmier	Trier (priorité P1–P5), placer/retourner, démarrer et compléter la consultation, gérer le cycle des lits, clôturer.
Médecin	Démarrer et clôturer la consultation, remplir le dossier clinique, demander transferts et relèves.
Chef de service	Superviser (SLA, alertes, blocages), administrer les équipes de salle, enregistrer avec override, gérer la capacité.
Administrateur	Gérer les utilisateurs, les rôles et la configuration système et IA.
Patient	Bénéficiaire ; son parcours est suivi et tracé.
Système IA	Assistant optionnel : propose priorité, flux de lits, capacité et documents, sans décider.

2.4 Besoins fonctionnels

2.4.1 Authentification et gestion des rôles

Connexion sécurisée par email et mot de passe, génération d'un jeton JWT, contrôle d'accès par rôle et par **capacités** (par exemple `register`, `call`, `place`, `triage`, `close`), gestion du cycle de vie des comptes.

2.4.2 Gestion des patients et des visites

Recherche/création du patient (identification par numéro national), création d'une visite avec code public unique (`V-AAAAMMJJ-XXXX`), contrainte d'une seule visite active

par patient, suivi multi-états (statut, phase de soin, orientation, localisation).

2.4.3 File d'attente et appel

File calculée et non stockée : position déterminée à la lecture par priorité, caractère critique puis ancienneté. L'appel ne retire pas le patient de la file ; il enregistre l'heure d'appel et un compteur d'appels.

2.4.4 Triage structuré

Saisie et historique des évaluations de triage (initiale, réévaluation, escalade) avec priorité P1–P5, note clinique, constantes et orientation ; pose d'un indicateur critique.

2.4.5 Gestion des ressources

Classification des ressources (lit, fauteuil, box, poste de triage, poste de consultation), cycle de vie opérationnel (prêt, nettoyage, maintenance, bloqué, hors service), affectation et libération, historique d'occupation.

2.4.6 Prise en charge et consultations

Démarrage de consultation, attribution du médecin, remplissage du dossier (diagnostic, notes), clôture avec outcome, examens et tâches de soin.

2.4.7 Orientation, transferts et relèves

Demandes de transfert de lit (pendant, confirmé, annulé, expiré) et demandes de relève entre soignants (acceptée, refusée, annulée), avec motif et traçabilité.

2.4.8 Supervision et alertes

Pipeline des patients, détection des blocages (patients « bloqués » au-delà de seuils SLA), alertes opérationnelles hiérarchisées (faible, moyenne, haute, critique) avec acquittement et résolution.

2.4.9 Assistance IA

Recommandations de triage, de flux de lits, de capacité et de documentation, avec traces d'exécution, décisions humaines et approbations.

2.5 Besoins non fonctionnels

Table 2.2 – Exigences non fonctionnelles

Exigence	Description
Intégrité	Toute action de visite est atomique (transaction PostgreSQL) ; les contraintes d'intégrité empêchent les états incohérents.
Concurrence	Le verrouillage (`SELECT ... FOR UPDATE`) empêche deux affectations simultanées du même lit.
Sécurité	JWT, hachage bcrypt, RBAC par capacités, validation serveur, protection des données patients.
Traçabilité	Chaque transition est journalisée dans `visit_events` avec acteur, horodatage et métadonnées.
Performance	Réponses API courtes, mises à jour temps réel sans rechargement.
Disponibilité	Service conçu pour un fonctionnement continu (24h/24).
Ergonomie	Tableaux de bord par rôle, interfaces intuitives, thème adapté au rôle.
Évolutivité	Architecture modulaire (services séparés), API REST extensible, migrations versionnées.
Idempotence	Reprise d'action sans doublon grâce à une clé d'idempotence.

2.6 Règles métier structurantes

Les règles suivantes gouvernent la cohérence du système et sont appliquées côté serveur, jamais uniquement côté interface :

1. **Une seule visite active par patient** ; un patient n'a jamais deux prises en charge ouvertes.
2. **Une seule affectation active par visite** et **un seul patient par ressource**.
3. **Transitions autorisées uniquement** : une action invalide pour l'état courant est rejetée (code HTTP 400/409).
4. **Compatibilité des ressources** : une ressource en nettoyage, maintenance, bloquée ou hors service ne peut être affectée.

5. **Tri obligatoire avant prise en charge** : la phase de soin suit une phase de triage explicite.
6. **Décision clinique humaine** : le triage et la clôture clinique ne sont jamais automatiques.
7. **Approbation humaine des actions IA** : toute action IA à risque exige une approbation, sauf outils explicitement autorisés.

2.7 Diagramme de cas d'utilisation

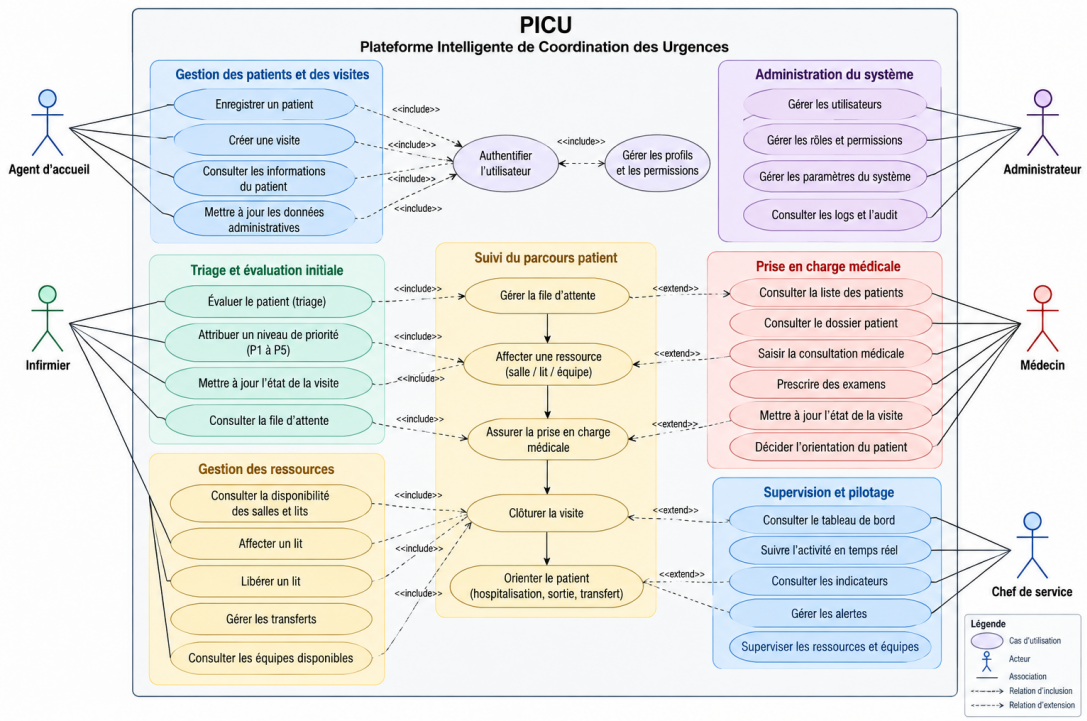


Figure 2.1 – Diagramme de cas d'utilisation de PICU

2.8 Description des principaux cas d'utilisation

2.8.0.1 UC1 — Authentification et redirection par rôle

Acteur : tous. Connexion par identifiants, émission d'un jeton JWT, redirection vers le tableau de bord du rôle.

2.8.0.2 UC2 — Enregistrement d'un patient

Acteur : Réceptionniste, Chef. Recherche par numéro national, création du patient et de la visite, priorité d'enregistrement et motif ; la visite entre en phase `triage_pending`.

2.8.0.3 UC3 — Triage clinique

Acteur : Infirmier. Saisie d'une évaluation de triage (priorité, note, constantes, orientation), historique conservé.

2.8.0.4 UC4 — Appel et placement

Acteur : Réceptionniste, Infirmier. Appel du patient dans la file, puis affectation à une ressource compatible et libre.

2.8.0.5 UC5 — Prise en charge médicale

Acteur : Médecin, Infirmier. Démarrage de consultation, remplissage du dossier, clôture avec outcome.

2.8.0.6 UC6 — Transfert et relève

Acteur : Médecin, Infirmier, Chef. Demande de transfert de lit ou de relève, confirmée ou refusée selon les rôles autorisés.

2.8.0.7 UC7 — Supervision

Acteur : Chef. Consultation du pipeline, des indicateurs SLA et des alertes opérationnelles ; acquittement et résolution.

2.8.0.8 UC8 — Assistance IA

Acteur : Système IA, personnel habilité. Production de recommandations et, le cas échéant, proposition d'actions soumises à approbation.

2.8.0.9 UC9 — Administration

Acteur : Administrateur. Gestion des utilisateurs, des rôles et de la configuration (dont les paramètres IA).

2.9 Matrice de traçabilité des exigences

La rigueur d'ingénierie exige de relier chaque exigence à son implémentation et à sa validation. Le tableau suivant en donne un extrait (la matrice complète figure en annexe).

Table 2.3 – Extrait de la matrice Exigence → Implémentation → Test

Exigence	**Implémentation**	**Validation**
Une seule visite active	`visitService.registerVisit`	Test d'intégrité / 409
Un seul patient par lit	`assertBedAssignable`, verrou	Test concurrence / 409
Transitions valides	`visitService`, `careStage`	Tests par rôle
Lit non affectable si « cleaning »	`resourceService`	`npm run test:v4`
Seuils SLA non approuvés inactifs	`supervisionRulesService`	`npm run test:supervision`
Indices d'enregistrement	`registrationHints`	`npm run test:registration`
Actions IA à risque approuvées	`action-policy`	Tests de politique IA

2.10 Gestion des risques

Table 2.4 – Risques identifiés et mesures de prévention

Risque	**Mesure de prévention**	**Limite résiduelle**
Erreur de priorisation	Assistance IA supervisée + historique de triage	Précision IA perfectible
Conflit d'affectation d'un lit	Verrou `FOR UPDATE` + contrainte d'affectation unique	Dépend de la disponibilité réelle
Perte de connexion temps réel	Raisonnement par état serveur + rechargement	Latence possible des écrans
Confidentialité des données	RBAC, JWT, secrets hors dépôt	Pas d'audit externe réalisé
Décision automatisée abusive	Politique d'actions IA, approbation obligatoire, mode simulation restreint	Nécessite gouvernance continue

2.11 Conclusion

Les besoins sont formalisés en exigences fonctionnelles et non fonctionnelles, règles métier, cas d'utilisation, matrice de traçabilité et analyse de risques. Le chapitre suivant présente la méthodologie de développement et la conception du système.

Chapitre 3

Méthodologie et conception du système

3.1 Introduction

Ce chapitre présente d'abord la **méthodologie de développement** (organisation, itérations, gestion de versions), puis la **conception** : architecture logicielle, machine à états, modélisation UML, conception de la base de données et choix technologiques. Conformément à l'objectif du rapport, chaque choix technique est justifié.

3.2 Méthodologie de développement

3.2.1 Démarche itérative et incrémentale

Le projet a été conduit par itérations successives, chacune livrant un incrément testable :

1. **Socle** : authentification, patients/visites, machine à états transactionnelle, boards ;
2. **Enrichissement du parcours (V3)** : phases de soin, orientations, historique de triage, cycle des lits, examens et tâches ;
3. **Supervision et ressources (V4)** : SLA, alertes, transferts, relèves, indicateurs de capacité ;
4. **Assistance IA** : agents de triage, lits, capacité et documentation, sous supervision.

3.2.2 Organisation du travail et jalons

Chaque incrément est matérialisé par une **migration SQL versionnée** (nommée `AAAAMMJJ_description.sql`), ce qui rend l'évolution du schéma reproductible et au-

ditables. Les jalons sont documentés dans le dépôt (plans, notes de passage de phase, rapports de test).

3.2.3 Gestion des versions

Le code est géré avec Git selon une convention de messages claire. Deux dispositifs garantissent l'hygiène du dépôt :

— des **hooks pre-commit** (scripts locaux) qui bloquent les erreurs évidentes ;

— un **pipeline GitHub Actions** « secret guard » qui échoue si un fichier `.env` est versionné.

3.3 Architecture logicielle

3.3.1 Architecture en trois couches

PICU suit une architecture client-serveur en trois couches :

1. **Présentation** : React 19 + Vite + TailwindCSS (tableaux de bord par rôle) ;
2. **Application** : Node.js + Express (API REST + Socket.IO), services métier et middleware ;
3. **Données** : PostgreSQL hébergé sur Neon.

3.3.2 Organisation des services backend

Le backend est organisé en couches pour séparer les responsabilités :

— **routes** (`*Routes.js`) : exposition HTTP, contrôle des capacités, validation d'entrée ;

— **services** (`*Service.js`) : logique métier transactionnelle (par exemple `visitService`, `resourceService`, `supervisionRulesService`, `transferRequestService`) ;

— **lib** : utilitaires transverses (par exemple `careStage`, journalisation d'actions) ;

— **middleware** : authentification (`authenticateJwt`), autorisation (`requireRole`, `requireCapabil`

— **ai** : agents, orchestrateur, fournisseur NIM, politiques d'action, observabilité.

3.3.3 Communication REST et temps réel

La communication combine des requêtes HTTP REST (JSON) et un canal temps réel **Socket.IO**. Le serveur diffuse des événements dérivés des transitions de visite (notamment `visit:status_changed` et `queue:updated`) afin que tous les postes se synchronisent sans rechargement.

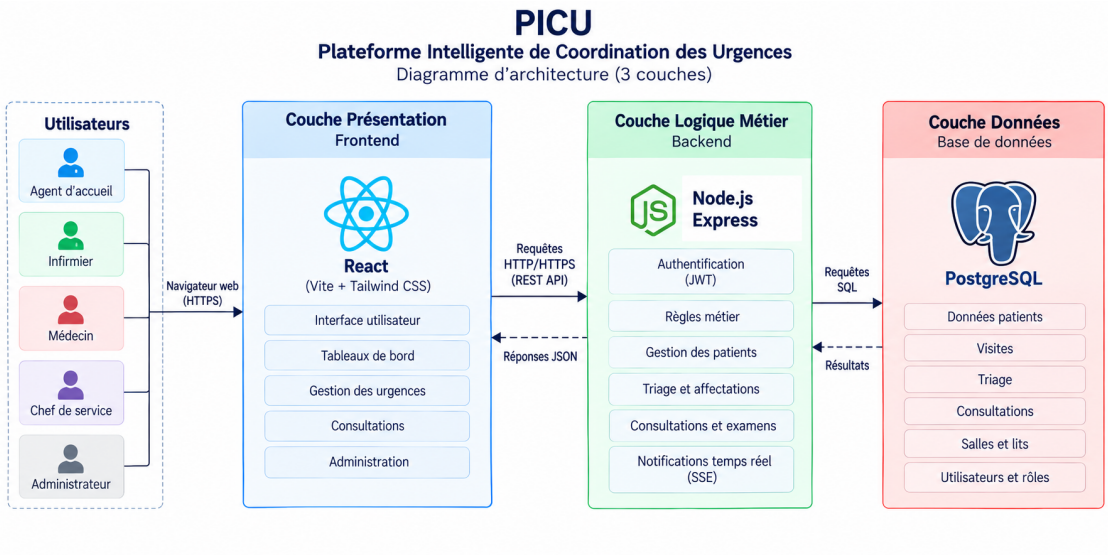


Figure 3.1 – Architecture en trois couches et communications REST/Socket.IO

3.4 Conception du workflow : machine à états

3.4.1 États de la visite

Le statut de la visite est la source de vérité du parcours. Il se décompose en trois états principaux et trois états terminaux.

Table 3.1 – Machine à états du statut de visite

Statut	Signification	Transitions sortantes
`waiting`	Enregistré, en attente de prise en charge	`place`, `close` (parti avant)
`placed`	Affecté à une ressource	`start`, `return`, `close`
`in_treatment`	Prise en charge en cours	`close`
`discharged`	Sortie autorisée (terminal)	—
`transferred`	Transféré (terminal)	—
`left_without_seen`	Parti avant prise en charge (terminal)	—

3.4.2 Actions et transitions

Chaque action est une transition atomique exécutée dans une transaction. Le tableau suivant résume les actions autorisées et leurs effets.

Table 3.2 – Actions de workflow et règles

Action	De	Vers	Effet principal
`register`	—	`waiting`	Crée patient + visite, une seule visite active
`call`	`waiting`	`waiting`	Horodate l'appel, incrémente le compteur
`place`	`waiting`	`placed`	Occupe une ressource libre et compatible
`return`	`placed`	`waiting`	Libère la ressource, remet en file
`start`	`placed`	`in_treatment`	Ouvre la consultation
`close`	`placed/in_treatment`	terminal	Libère la ressource, fixe l'outcome
`triage`	`waiting`	`waiting`	Enregistre une évaluation de triage

3.4.3 Dimension multi-états

Au-delà du statut, la visite porte des dimensions orthogonales qui permettent un suivi fin : la **phase de soin** (`care_stage` : `registered`, `triage_pending`, `waiting_care`, `in_care`, `in_observation`, `disposition_pending`, `completed`), l'**orientation** (`critical`, `acute`, `standard`, `ambulatory`, `observation`, `isolation`), la **localisation** du patient et le **code public**.

3.4.4 Conception de la concurrence

Le point critique est l'affectation des ressources. La solution retenue combine :

1. un verrou de ligne sur la visite (`SELECT ... FOR UPDATE`) ;
2. une vérification qu'aucune affectation active n'existe pour la ressource ni pour la visite ;
3. une insertion d'affectation et une mise à jour de statut dans la **même transaction**.

Ainsi, deux infirmiers qui tentent d'affecter le même lit simultanément voient l'un réussir et l'autre échouer avec un conflit (HTTP 409), sans jamais produire d'état incohérent.

3.5 Diagrammes UML

3.5.1 Diagramme de séquence — enregistrement

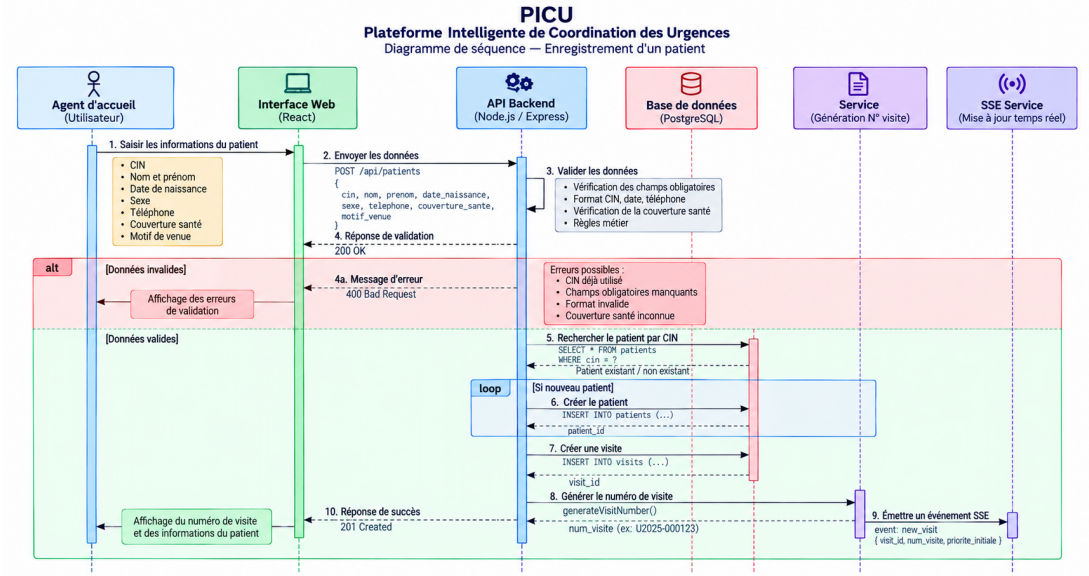


FIGURE 3.2 – Diagramme de séquence — Enregistrement d'un patient

3.5.2 Diagramme d'activité — circuit de triage

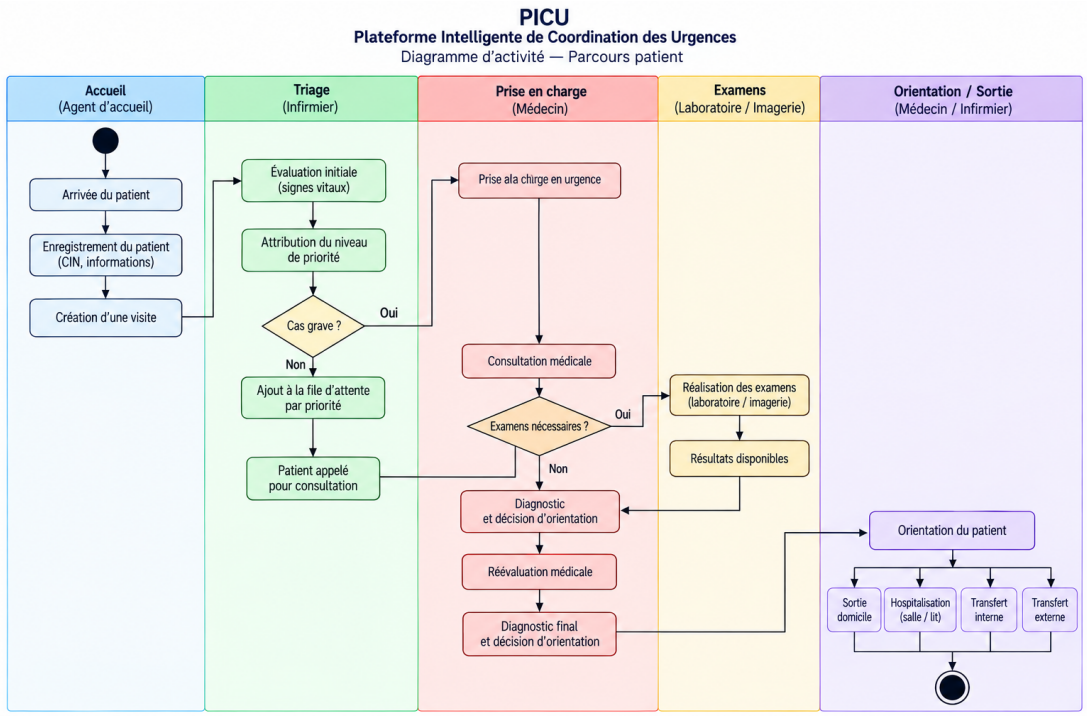


FIGURE 3.3 – Diagramme d'activité — Circuit de triage et de prise en charge

3.5.3 Diagramme de classes

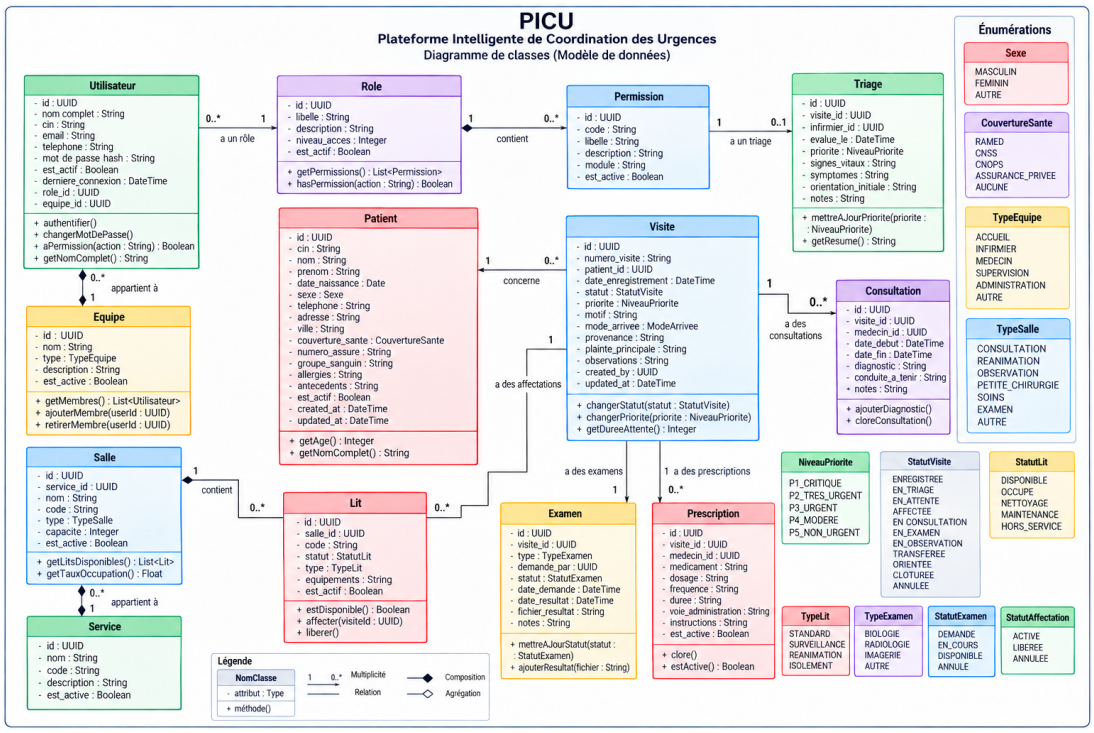


Figure 3.4 – Diagramme de classes de PICU

3.6 Conception de la base de données

3.6.1 Modèle relationnel

La base repose sur un schéma relationnel normalisé (3FN). Les entités principales sont : `users`, `patients`, `visits`, `zones`, `rooms`, `beds`, `bed_assignments`, `consultations`, `triage_assessments`, `exams`, `care_tasks`, `visit_events`, `room_teams`, `resource_status_events`, ainsi que les tables V4 (`transfer_requests`, `handoff_requests`, `operational_alerts`, `workflow_settings`) et IA (`agent_runs`, `agent_recommendations`, `agent_actions`, `agent_approvals`

3.6.2 Contraintes d'intégrité

L'intégrité est assurée au niveau du schéma par :

— des **clés étrangères** avec politiques de suppression restrictives pour les entités critiques ;

— des **contraintes de domaine** (`CHECK`) sur les statuts, phases, orientations, outcomes et statuts opérationnels ;

— des **index uniques** empêchant un doublon de code public ou d'affectation active ;

— des **vues** (`waiting_board`, `bed_board`) exposant l'état consolidé.

3.6.3 Dictionnaire de données (extrait)

Table 3.3 – Dictionnaire de données — extrait

Table	Colonne	Description
visit_events	event_type	Type d'événement (enregistré, appelé, placé, triage, clôturé...)
	actor_type	`user`, `agent` ou `system`
	idempotency_key	Clé unique garantissant la non-duplication d'une action
visits	status	Statut de la machine à états
	care_stage	Phase de soin
	current_orientation	Orientation clinique courante
beds	resource_type / operational_status	Type (lit, fauteuil, box...) et état opérationnel
triage_assessments	assessment_type	`initial`, `reassessment` ou `escalation`
transfer_requests	status	`pending`, `confirmed`, `cancelled`, `expired`
agent_actions	requires_approval	Indique si l'action IA doit être approuvée

3.6.4 Stratégie de migration

Chaque évolution est apportée par une migration additive et idempotente (`ADD COLUMN IF NOT EXISTS`, `CREATE TABLE IF NOT EXISTS`) appliquée par un script dédié (`npm run migrate:v3`, `migrate:v4`, `migrate:ai`). Cette stratégie permet de faire évoluer un schéma en production sans rupture.

3.7 Choix technologiques et justification

Table 3.4 – Choix technologiques et justification

Brique	**Technologie**	**Justification**
Frontend	React 19 + Vite	Écosystème mature, composants réutilisables, build rapide.
Style	TailwindCSS	Cohérence visuelle et rapidité de prototypage.
Backend	Node.js + Express	Même langage que le frontend, middleware simple.
Base	PostgreSQL (Neon)	Intégrité relationnelle, transactions, JSONB, gestion managée.
Temps réel	Socket.IO	Diffusion d'événements, repli polling.
Auth	JWT + bcrypt	Standard, sans session serveur.
IA	NVIDIA NIM	Modèles spécialisés, sorties structurées (JSON schema).

3.8 Conclusion

La méthode itérative, une architecture en couches, une machine à états explicite et une base fortement contrainte constituent le socle de PICU. Le chapitre suivant détaille la réalisation concrète de ces choix.

Chapitre 4

Réalisation de la plateforme PICU

4.1 Introduction

Ce chapitre décrit la réalisation concrète des choix de conception. Il présente l'environnement de développement, puis chaque module du parcours patient, les interfaces, la communication temps réel et la couche d'assistance IA. Pour chaque module, la logique métier et sa traduction technique sont explicitées.

4.2 Environnement de développement

4.2.1 Technologies

React 19, Vite, TailwindCSS, React Router, Node.js, Express, PostgreSQL (Neon), Socket.IO, JWT, bcrypt, NVIDIA NIM (SDK compatible OpenAI), `node:test`.

4.2.2 Outils

Visual Studio Code, Git, GitHub Actions, Chrome/Firefox (DevTools), Draw.io / Mermaid, pgAdmin/psql.

4.3 Module d'authentification et de gestion des accès

L'authentification est implémentée dans les routes d'authentification et le middleware `authenticateJwt`. Le mot de passe est haché avec bcrypt ; la connexion émet un jeton JWT stocké côté client. À chaque requête, le middleware recharge l'utilisateur en base et rejette tout compte inactif. L'autorisation combine deux niveaux : un contrôle par **rôle** (`requireRole`) et un contrôle plus fin par **capacité** (`requireCapability`).

Table 4.1 – Capacités par rôle (extrait)

Rôle	**Capacités**
Réceptionniste	register, call, critical
Infirmier	place, return, start, fill, close, triage, critical, ai_documentation, ai_triage_recommend, ai_bed_flow
Médecin	start, close, fill, ai_documentation, ai_triage_recommend
Chef	register, call, place, start, return, close, fill, critical, ai_health, ai_settings, + IA
Administrateur	manage_users, ai_health, ai_settings

À insérer : `images/login_screen.png`

Figure 4.1 – Page de connexion de PICU

4.4 Module patients et visites

Les routes de visite exposent les actions du workflow. L'enregistrement crée le patient s'il est inconnu (recherche par numéro national), vérifie qu'aucune visite active n'existe, puis crée la visite avec un **code public unique** et la place en phase de triage. Un mécanisme d'**idempotence** (clé fournie par le client) garantit qu'un double envoi ne crée pas deux visites.

Chaque action suit le même patron : ouverture d'une transaction, verrouillage de la visite, contrôle de l'état courant, écriture métier, journalisation d'un événement `visit_events`, puis validation de la transaction.

4.5 Module file d'attente et appel

La file d'attente n'est pas stockée comme une table mutable : elle est **calculée** à la lecture par une vue (`waiting_board`) qui ordonne les patients par priorité croissante, indicateur critique puis date d'arrivée. La position est donc cohérente sans risque de doublons.

L'action `call` reste dans l'état `waiting` : elle enregistre l'heure du dernier appel et incrémente un compteur, matérialisant visuellement qu'un patient « a été appelé » sans le retirer de la file. Le staff de chaque rôle dispose de vues de travail (`/queue/worklists`) filtrées selon son périmètre.

4.6 Module de triage

Le triage est structuré et historisé. Chaque évaluation est enregistrée dans `triage_assessments` avec sa priorité (1–5), son type (`initial`, `reassessment`, `escalation`), une note clinique, des constantes éventuelles (JSONB), une orientation et, en cas de changement, un motif. Un indicateur `critical_flag` peut être posé pour le fast-track des patients critiques.

Le système distingue explicitement la **priorité d'enregistrement** (fixée à l'accueil) et la **priorité clinique** (validée au triage), ce qui permet de mesurer l'apport du triage et d'éviter de confondre les deux notions.

4.7 Module de gestion des ressources

4.7.1 Typologie et cycle de vie

Une ressource est un lit typé (`resource_type` : lit, fauteuil, box, poste de triage, poste de consultation) associé à une salle et à une zone. Elle possède un **état opérationnel** (`ready`, `cleaning`, `maintenance`, `blocked`, `out_of_service`). Les transitions d'état sont tracées dans `resource_status_events`, et le passage direct de « nettoyage » à « prêt » est refusé pour imposer un contrôle humain de remise en service.

4.7.2 Affectation et libération

L'affectation vérifie que la ressource est affectable (`assertBedAssignable`), qu'aucune affectation active n'existe, puis crée l'affectation et passe la visite en `placed`. À la clôture ou au retour, la ressource est libérée et bascule en **turnover** (nettoyage) avant remise à disposition.

4.7.3 Le plan des lits

L'interface `BedPlanPanel` présente les zones, salles et lits avec leur occupation, et permet l'affectation d'un lit libre. Elle est partagée par l'infirmier, le médecin et le chef, avec des droits différents.

À insérer : `images/dashboard_infirmier.png`

Figure 4.2 – Dashboard Infirmier — triage, plan des lits et file

4.8 Module de prise en charge et consultations

Le démarrage d'une consultation crée une ligne `consultations` (médecin attribué, auteur de la saisie), passe la visite en `in_treatment` et en phase `in_care`. Le remplissage met à jour diagnostic et notes ; la clôture libère la ressource, clôture la consultation et fixe l'**outcome** (sortie, hospitalisation, transfert, parti avant ou pendant la prise en charge). Les règles d'accès garantissent qu'un médecin ne gère que les patients de sa salle et qu'un infirmier ne rédige que pour le médecin référent de la salle.

À insérer : `images/dashboard_medecin.png`

Figure 4.3 – Dashboard Médecin — patients actifs et consultations

4.9 Module transferts, relèves, examens et tâches

4.9.1 Transferts de lit

Une demande de transfert (`transfer_requests`) décrit le lit source, le lit cible et un statut (`pending`, `confirmed`, `cancelled`, `expired`). La confirmation n'est autorisée qu'à certains rôles et déclenche le déplacement effectif, tracé par un événement.

4.9.2 Relèves

Une demande de relève (`handoff_requests`) formalise le passage de responsabilité d'un soignant à un autre, avec motif et statut (`pending`, `accepted`, `declined`, `cancelled`).

4.9.3 Examens et tâches de soin

Les tables `exams` et `care_tasks` permettent de demander des examens (biologie, imagerie) et de créer des tâches de soin, avec statuts et horodatages, l'ensemble étant journalisé et visible dans le dossier de la visite.

4.10 Module supervision et alertes

Le chef de service dispose d'un centre de commande (*command center*) qui agrège :

— le **pipeline** des patients par étape ;

— les **indicateurs SLA** et la détection de patients bloqués ;

— les **alertes opérationnelles** hiérarchisées ;

— les indicateurs de **capacité** et de charge ;

— les **équipes de salle** et le plan des lits.

4.10.1 Seuils SLA

Les seuils de blocage sont configurables ; les valeurs par défaut sont présentées ci-dessous. Chaque dépassement est qualifié de *warning* ou de *critical* selon un facteur deux.

Table 4.2 – Seuils SLA par défaut (chef de service)

Situation	**Seuil (minutes)**
Attente non appelé	20
Appelé sans mise au lit	15
Au lit sans consultation	30
Prise en charge prolongée	60

4.10.2 Règles de supervision

Les règles de supervision sont pilotées par des paramètres (`workflow_settings`). Par prudence, une règle de délai n'est active que si elle est explicitement **approuvée** (le test `ruleConfigured` le vérifie). Les règles de réévaluation périodique et de pression de capacité sont activées ; les seuils de délai non validés restent désactivés en attente de validation par l'établissement.

4.10.3 Alertes opérationnelles

Les alertes (`operational_alerts`) possèdent un cycle de vie (`open`, `acknowledged`, `resolved`, `obsolete`), une sévérité, une déduplication par clé et un historique d'événements. Leur évaluation est déclenchée périodiquement par le serveur.

À insérer : `images/dashboard_chef.png`

Figure 4.4 – Dashboard Chef de service — centre de commande

4.11 Tableaux de bord par rôle

Chaque rôle dispose d'un tableau de bord dédié :

— **Accueil** : enregistrement, file, KPI, appel ;

— **Infirmier** : triage, plan des lits, file, tâches ;

— **Médecin** : charge de travail, salle, consultations ;

— **Chef** : pipeline, SLA, alertes, capacité, équipes ;

— **Administrateur** : utilisateurs, rôles, configuration.

À insérer : `images/dashboard_receptionniste.png`

Figure 4.5 – Dashboard Accueil — file d'attente et enregistrement

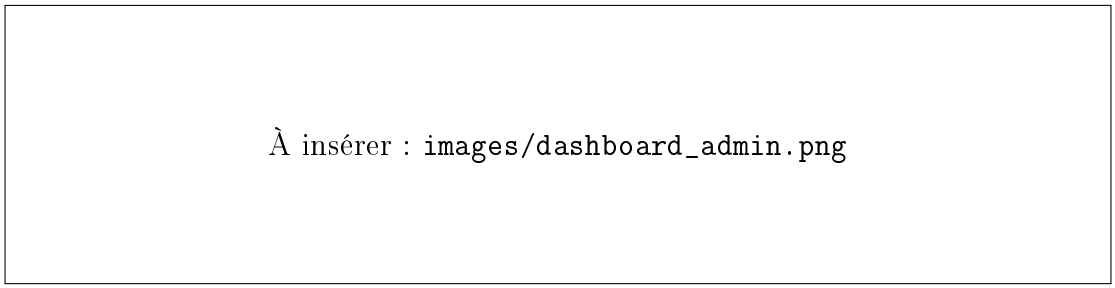


FIGURE 4.6 – Dashboard Administrateur — configuration du système

4.12 Synchronisation temps réel

À chaque transition importante, le backend publie un événement Socket.IO. Les interfaces écoutent ces événements et se rafraîchissent sans rechargement de page, un **repli par interrogation périodique** assurant la résilience en cas de coupure du canal.

TABLE 4.3 – Principaux événements temps réel

Événement	**Déclencheur**
`visit:status_changed`	Enregistrement, placement, prise en charge, clôture
`queue:updated`	Appel, placement, retour en file, nouvelle arrivée
`visit:timeline_updated`	Nouvel événement de visite (audit)
`resource:status_changed`	Changement d'état d'une ressource (nettoyage, prêt)

4.13 Couche d'assistance IA

4.13.1 Objectifs et principes

La couche IA vise à assister les équipes sans jamais décider à leur place. Quatre intentions sont supportées : **triage**, **flux de lits**, **capacité** et **documentation**. Chaque appel produit une exécution tracée (`agent_runs`) et une recommandation (`agent_recommendations`).

4.13.2 Fournisseur et routage des modèles

L'accès aux modèles passe par NVIDIA NIM. Un routeur de modèles associe à chaque tâche un profil : *reasoning* pour le triage et la documentation, *fast* pour les lits et la capacité, *embedding* pour la recherche documentaire. Les sorties sont contraintes par des schémas JSON.

4.13.3 Mode d'exécution et supervision

Le mode IA est globalement configurable : `disabled`, `collaborative`, `supervised_autonomy`, `full_control_simulation`. Des indicateurs par agent (documentation, triage, triage « shadow », lits, capacité) permettent d'activer sélectivement chaque fonction.

4.13.4 Politique d'action et approbations

Toute action IA est soumise à une **politique de risque** : les actions cliniques (triage, clôture) sont à risque maximal et exigent une approbation humaine ; seules quelques actions à faible risque (créer une tâche de réévaluation, rafraîchir une liste, lever une alerte) sont automatiques en mode supervisé. Un garde-fou interdit le mode « simulation » sur des visites réelles.

Table 4.4 – Politique d'actions IA (extrait)

Outil	**Risque**	**Approbation requise**
triage	critique	oui
close	critique	oui
place / transfer	haut	oui
call / return	moyen	oui
createReassessmentTask	faible	non (mode supervisé)
raiseOperationalAlert	faible	non (mode supervisé)

4.13.5 Mode « shadow » du triage

Pour évaluer le triage IA sans risque, un mode « shadow » permet d'exécuter une recommandation en arrière-plan lors de l'enregistrement, sans l'afficher comme décision officielle. Cela permet de comparer, hors production, la proposition IA à la décision humaine.

4.14 Conclusion

L'ensemble du parcours patient est implémenté, de l'accueil à la clôture, avec des modules avancés de supervision, de ressources et d'assistance IA. Le chapitre suivant traite de l'assurance qualité, de la sécurité et de l'industrialisation.

Chapitre 5

Assurance qualité, sécurité et CI/CD

5.1 Introduction

Ce chapitre traite d'un axe souvent négligé dans les rapports de stage : la **vérification de la qualité**, la **sécurité** et l'**industrialisation**. Nous présentons la stratégie de test et les résultats réellement obtenus, les mécanismes de sécurité et d'intégrité, puis la chaîne d'intégration et de livraison continue.

5.2 Stratégie de test

5.2.1 Niveaux de test

La stratégie combine plusieurs niveaux complémentaires :

— **tests unitaires** : fonctions pures de règles (supervision, indices d'enregistrement) ;

— **tests d'intégrité** : invariants de ressources et transitions, exécutés contre une base isolée ;

— **tests fonctionnels** : parcours par rôle (accueil, infirmier, médecin, chef) ;

— **tests de concurrence** : affectation simultanée d'une même ressource ;

— **tests de bout en bout** : scénario complet du parcours patient.

5.2.2 Outillage

Les tests automatisés s'appuient sur le lanceur natif de Node.js (`node:test`) et `node:assert`, sans dépendance à un framework externe. Ils sont exécutés via des scripts npm (`test:v4`, `test:supervision`, `test:registration`). Un *benchmark* de triage (`benchmarks/run-triage-benchma` mesure la latence du modèle NIM et enregistre ses résultats.

5.3 Tests automatisés et résultats

5.3.1 Invariants de ressources — `npm run test:v4`

Ce test vérifie trois invariants du cycle de vie des lits :

1. une ressource en **nettoyage** ne peut être affectée (rejet HTTP 409) ;
2. la libération d'une ressource la bascule en **nettoyage** (et non directement en « prêt ») ;
3. la transition directe vers « prêt » est refusée (HTTP 422), imposant le contrôle de remise en service.

5.3.2 Règles de supervision — `npm run test:supervision`

Ce test vérifie que `ruleConfigured` n'active **jamais** un seuil de délai qui n'est pas approuvé : une règle activée sans approbation est considérée inactive, garantissant qu'aucune alerte SLA non validée ne soit émise.

5.3.3 Intelligence d'enregistrement — `npm run test:registration`

Ce test vérifie que les **indices d'enregistrement** signalent un motif à risque, une arrivée en ambulance ou une priorité d'enregistrement élevée, sans jamais modifier la priorité clinique.

5.3.4 Analytique patient

Le module analytique dispose de son propre test vérifiant la cohérence des agrégats.

Table 5.1 – Matrice de tests — extrait

ID	Scénario	Résultat attendu	Statut
T1	Affecter un lit en nettoyage	Rejet (409)	OK
T2	Libérer un lit	Passe en nettoyage	OK
T3	Forcer « prêt » sans contrôle	Rejet (422)	OK
T4	Seuil SLA non approuvé	Inactif	OK
T5	Indice motif critique	Suggestion sans mutation	OK
T6	Double affectation du même lit	Un succès, un conflit (409)	OK
T7	Double enregistrement (idempotence)	Une seule visite	OK
T8	Parcours E2E complet	Transitions cohérentes	OK

5.4 Cas d'étude : concurrence sur l'affectation d'un lit

5.4.1 Le problème

Deux infirmiers consultent un même lit libre et cliquent « Affecter » au même instant. Sans précaution, deux patients pourraient se retrouver sur le même lit.

5.4.2 La solution retenue et sa vérification

L'affectation s'exécute dans une transaction : la visite est verrouillée (`SELECT ... FOR UPDATE`), l'absence d'affectation active est vérifiée, puis l'affectation est insérée et le statut mis à jour. Deux transactions concurrentes sont sérialisées par le verrou : la seconde détecte l'occupation et échoue proprement avec un conflit. Ce comportement est décrit par le test T6.

5.5 Tests fonctionnels et bout en bout

Les parcours par rôle ont été validés manuellement et par scénario : enregistrement à l'accueil, triage infirmier, appel et placement, prise en charge et clôture. Le scénario de référence (documenté dans le dépôt) vérifie que chaque transition produit l'événement attendu dans `visit_events`, que la phase de soin reste cohérente et qu'aucune erreur 4xx/5xx inattendue ne survient.

5.6 Gestion des erreurs

Le backend normalise les erreurs : les collisions d'unicité et de clé étrangère PostgreSQL sont converties en réponses HTTP claires (409/400), et les erreurs du fournisseur IA (délai, sortie invalide, erreur HTTP) sont traduites en codes explicites. Aucune trace technique brute n'est renvoyée au client.

5.7 Sécurité et intégrité

5.7.1 Contrôle d'accès

L'accès repose sur deux niveaux : le rôle et la **capacité**. Un point d'entrée vérifie systématiquement la capacité requise, de sorte qu'un rôle ne peut exécuter que les actions autorisées, indépendamment de l'interface.

5.7.2 Sécurité des échanges et des secrets

Les échanges sont authentifiés par jeton JWT ; les mots de passe sont hachés. Les secrets (chaîne de connexion, clé JWT, clé du fournisseur IA) sont placés dans des fichiers `.env` **jamais versionnés** ; un pipeline échoue si un tel fichier est committé. Les en-têtes de sécurité (Helmet) et CORS sont configurés.

5.7.3 Intégrité et traçabilité

L'intégrité est garantie par les transactions et les contraintes SQL. La traçabilité repose sur `visit_events`, qui enregistre pour chaque action l'acteur (utilisateur, agent ou système), l'horodatage, les états source et cible et les métadonnées. Cette piste constitue la base de l'audit et de la conformité.

5.7.4 Accès aux données patients

L'accès est restreint par rôle, et le service ciblé vérifie en plus que le soignant est bien rattaché à la salle du patient. Les données restent sous le contrôle du personnel autorisé.

5.8 Qualité du code

La qualité est soutenue par : une organisation en couches (séparation routes/services/lib), une convention de nommage (`*Routes.js`, `*Service.js`), un linter (Oxlint) côté frontend, la journalisation structurée des actions de visite et des messages de commit lisibles.

5.9 Industrialisation et CI/CD

5.9.1 Gestion du code source

Le dépôt est géré avec Git. Un fichier `.gitignore` racine exclut les dépendances et les secrets. Des **hooks pre-commit** fournissent une première barrière locale avant tout commit.

5.9.2 Intégration continue

Un workflow GitHub Actions, « secret guard », s'exécute à chaque `push` et `pull request`. Il vérifie qu'aucun fichier `.env` (hors `.example`) n'est suivi par Git.

Listing 5.1 – Extrait du pipeline GitHub Actions « secret guard »

```yaml
name: Secret guard
on:
  push:
    branches: [main, master]
  pull_request:

jobs:
  block-env-files:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Fail if .env files are tracked
        run: |
          bad=$(git ls-files | grep -E '(^|/)\.env(\.|$)' \
            | grep -Ev '\.env\.example$' || true)
          if [ -n "$bad" ]; then
            echo "::error::Tracked env files must not be in the repo:"
            echo "$bad"; exit 1
          fi
```

5.9.3 Environnements et livraison

Deux environnements coexistent : le développement local (frontend Vite, backend Node, base Neon managée) et l'environnement de démonstration. La livraison d'un nouvel incrément suit une procédure reproductible : application des migrations SQL versionnées, puis démarrage du serveur.

5.9.4 Limites et améliorations de la CI/CD

Le pipeline actuel se concentre sur la protection des secrets. Son extension **proposée** comprendrait l'exécution automatique des tests, la vérification du lint et la construction du frontend à chaque `pull request`, ainsi qu'un déploiement automatisé. Cette extension est présentée comme une évolution et non comme un acquis.

5.10 Conclusion

La qualité est vérifiée par des tests explicites et des cas d'étude (concurrence, idempotence), la sécurité repose sur le RBAC, JWT et la traçabilité, et l'industrialisation est assurée par Git, les hooks et un pipeline minimal. Le chapitre suivant dresse l'évaluation, le bilan et les perspectives.

Chapitre 6

Évaluation, bilan et perspectives

6.1 Introduction

Ce chapitre évalue le projet au regard des objectifs initiaux, identifie les difficultés rencontrées et les solutions adoptées, expose les limites et les compétences acquises, puis trace les perspectives d'évolution, notamment l'assistance IA NVIDIA NIM.

6.2 Évaluation du projet

6.2.1 Fonctionnalités réalisées et état de validation

Le tableau suivant confronte les objectifs initiaux aux fonctionnalités effectivement réalisées et à leur état de validation, en distinguant ce qui est mature, opérationnel en démonstration et préparatoire.

Table 6.1 – Objectifs, réalisation et validation

Objectif	**Réalisation**	**État**
Workflow transactionnel	Machine à états, actions atomiques, audit	Mature, testé
File d'attente fiable	File calculée, appel non destructif	Mature
Triage structuré	Historique de triage, orientations, flag critique	Mature
Gestion des ressources	Cycle de vie des lits, verrous, plan	Mature, testé
Transferts et relèves	Demandes avec statuts et traçabilité	Opérationnel (démo)
Supervision SLA	Seuils, alertes, blocages	Opérationnel (démo)
Examens et tâches	Tables et endpoints dédiés	Opérationnel (démo)
Temps réel	Socket.IO + repli polling	Mature
Assistance IA	Agents, approbations, mode shadow	Préparatoire / supervisé
CI/CD	Hooks, pipeline secret guard	Mature (minimal)

6.2.2 Performance et fiabilité

Les temps de réponse de l'API sont courts et les mises à jour temps réel suppriment le besoin de rechargement. La fiabilité est démontrée par les tests d'intégrité et le cas de concurrence. La précision du triage IA n'est pas présentée ici comme un résultat clinique validé : elle relève d'une évaluation en cours et reste perfectible.

6.3 Difficultés rencontrées et solutions apportées

Table 6.2 – Difficultés et solutions

Difficulté	**Solution adoptée**
Coordonner des actions concurrentes	Transactions + verrous `FOR UPDATE`, une affectation active par ressource.
Éviter les doublons d'action	Clés d'idempotence uniques au niveau des événements.
Faire évoluer un schéma en continu	Migrations additives et idempotentes, scripts dédiés par phase.
Séparer décisions cliniques et automatisation	Politique d'actions IA, approbations, mode shadow.
Sécuriser des données sensibles	RBAC par capacités, JWT, secrets hors dépôt, audit.
Garder tous les postes synchronisés	Socket.IO avec repli par interrogation périodique.

6.4 Limites de la plateforme

— **Pas de déploiement clinique** : la plateforme est évaluée hors production, sans validation médicale officielle ;

— **précision du triage IA** perfectible, justifiant la supervision humaine ;

— **CI/CD partielle** : le pipeline couvre la protection des secrets, pas encore l'exécution automatisée des tests et le déploiement ;

— **couverture de tests** inégale selon les modules ;

— **absence d'intégration** avec le système d'information hospitalier (SIH).

6.5 Compétences acquises

Ce stage a permis de mobiliser et de consolider : la modélisation des processus métier (machine à états, UML), la conception de bases de données relationnelles contraintes, le développement d'une API REST sécurisée, la gestion de la concurrence, la traçabilité applicative, la mise en place d'une stratégie de tests et les bases de l'industrialisation logicielle. Sur le plan humain, il a renforcé la capacité à dialoguer avec des professionnels de santé pour traduire des besoins cliniques en exigences techniques.

6.6 Perspectives

6.6.1 Assistance IA NVIDIA NIM

L'évolution la plus prometteuse est le renforcement et la validation progressive de la couche d'assistance IA. Les axes envisagés : enrichir le contexte de triage, industrialiser l'évaluation en mode shadow, mesurer la précision priorité et orientation sur un jeu de cas cliniques, et étendre le périmètre des agents (documentation, flux de lits, capacité) sous gouvernance stricte.

6.6.2 Industrialisation complète

Étendre la CI/CD : exécution automatique de `test:v4`, `test:supervision` et `test:registration` à chaque `pull request`, vérification du lint, construction du frontend et déploiement automatisé vers un environnement de recette.

6.6.3 Interopérabilité hospitalière

Préparer l'interopérabilité avec le SIH via des standards comme HL7/FHIR, afin d'échanger les admissions, les identités et les comptes rendus.

6.6.4 Indicateurs et pilotage

Enrichir la supervision d'indicateurs avancés : taux d'acceptation des recommandations IA, taux de réaffectation, temps d'attente par zone et par priorité, taux d'occupation des lits.

6.6.5 Chemin vers une validation clinique

Structurer une démarche de validation : définition d'indicateurs de sécurité, revue par les équipes médicales, protocole d'évaluation en conditions contrôlées et gouvernance des données.

6.7 Conclusion

Le projet atteint ses objectifs d'ingénierie : un workflow explicite, fiable et traçable, des interfaces par rôle, une supervision opérationnelle et une assistance IA supervisée. Ses limites, honnêtement exposées, tracent une feuille de route claire vers l'industrialisation, l'interopérabilité et, à terme, une validation clinique encadrée.

Conclusion générale

Le projet PICU (Plateforme Intelligente de Coordination des Urgences) avait pour ambition de répondre aux difficultés organisationnelles du service des urgences du CHU Hassan II de Fès. Au terme de ce stage, la contribution principale ne réside pas dans l'ajout d'un écran, mais dans la formalisation d'un **workflow transactionnel explicite** : une machine à états de la visite, des transitions autorisées, des priorités cliniques structurées, des files d'attente calculées et une gestion des ressources verrouillée en base de données.

La plateforme réalisée couvre l'ensemble du parcours patient, de l'accueil à la clôture, complétée par des modules avancés : triage structuré et historisé, cycle de vie des lits, demandes de transfert et de relève, examens et tâches de soin, supervision opérationnelle avec indicateurs SLA et alertes, et tableaux de bord par rôle. La communication temps réel, assurée par Socket.IO avec un repli par interrogation périodique, garantit une vision partagée entre tous les postes.

L'accent mis sur l'ingénierie logicielle se traduit par des choix vérifiables : transactions PostgreSQL et verrous pour prévenir les conflits d'affectation, clés d'idempotence contre les doublons, contrôle d'accès par rôle et capacités, piste d'audit exhaustive, tests automatisés des invariants critiques et un pipeline d'intégration continue protégeant les secrets. Les cas d'étude de concurrence et d'intégrité constituent une preuve concrète de la robustesse du socle.

La couche d'assistance IA fondée sur NVIDIA NIM, tenue volontairement sous supervision humaine, ouvre une perspective prometteuse : assister le triage et l'organisation des lits sans jamais se substituer à la décision clinique. Sa validation rigoureuse, en mode shadow puis encadré, constitue la prochaine étape naturelle.

Enfin, ce rapport s'est attaché à distinguer clairement ce qui est **réalisé et évalué** de ce qui demeure **exploratoire**, condition indispensable pour une application de santé. Les limites identifiées — absence de déploiement clinique, CI/CD à compléter, interopérabilité SIH à bâtir — dessinent une feuille de route cohérente vers une plateforme plus complète et, à terme, validée par les équipes médicales.

Ce stage a confirmé l'importance d'une démarche d'ingénierie structurée et du dialogue entre informaticiens et professionnels de santé pour concevoir des solutions véritablement

adaptées au terrain clinique.

Bibliographie

[1] Express.js Foundation. *Express.js - Node.js web application framework*, 2024. Documentation officielle.

[2] OpenJS Foundation. *Node.js - JavaScript runtime built on Chrome's V8 JavaScript engine*, 2024. Documentation officielle.

[3] PostgreSQL Global Development Group. *PostgreSQL - The World's Most Advanced Open Source Relational Database*, 2024. Documentation officielle.

[4] IETF. *JSON Web Token (JWT)*, 2015. RFC 7519.

[5] Tailwind Labs. *Tailwind CSS - A utility-first CSS framework*, 2024. Documentation officielle.

[6] Meta. *React - A JavaScript library for building user interfaces*, 2024. Documentation officielle.

[7] Neon. *Neon - Serverless Postgres*, 2025. Documentation officielle.

[8] NVIDIA. *NVIDIA NIM - Inference Microservices*, 2025. Documentation officielle.

[9] Socket.IO. *Socket.IO - Bidirectional and low-latency communication*, 2024. Documentation officielle.

[10] React Training. *React Router - Declarative routing for React*, 2024. Documentation officielle.

[11] Evan You. *Vite - Next Generation Frontend Tooling*, 2024. Documentation officielle.

Webographie

— React Documentation — https://react.dev/

— Node.js Documentation — https://nodejs.org/docs/

— Node.js Test runner — https://nodejs.org/api/test.html

— Express.js — https://expressjs.com/

— PostgreSQL Documentation — https://www.postgresql.org/docs/

— PostgreSQL — Concurrency Control (verrous) — https://www.postgresql.org/docs/current/mvcc.html

— Socket.IO — https://socket.io/docs/

— TailwindCSS — https://tailwindcss.com/docs

— Vite — https://vitejs.dev/

— React Router — https://reactrouter.com/

— Neon (PostgreSQL managé) — https://neon.tech/docs

— NVIDIA NIM — https://docs.nvidia.com/nim/

— GitHub Actions — https://docs.github.com/actions

— Draw.io (diagrams.net) — https://app.diagrams.net/

— HL7 FHIR — https://hl7.org/fhir/

Annexes

6.8 Annexe A — Diagrammes UML

Diagrammes de cas d'utilisation, de séquence, d'activité et de classes réalisés avec Draw.io / Mermaid. Voir figures 2.1, 3.2, 3.3 et 3.4 (chapitres 2 et 3).

6.9 Annexe B — Modèle de base de données

Le schéma PostgreSQL est décrit par les migrations versionnées du dossier `backend/migrations/`, appliquées par des scripts dédiés. Le dictionnaire de données est présenté au tableau 3.3 (chapitre 3).

6.10 Annexe C — Documentation des API

Les points d'entrée REST sont regroupés par domaine. Extrait des principaux endpoints :

- `POST /api/auth/login` — authentification (JWT) ;
- `POST /api/visits/register` — enregistrement patient + visite ;
- `POST /api/visits/:id/call` — appel du patient ;
- `POST /api/visits/:id/place` — affectation à une ressource ;
- `POST /api/visits/:id/start` — démarrage de la consultation ;
- `PATCH /api/visits/:id/priority` — triage / priorité ;
- `POST /api/visits/:id/close` — clôture avec outcome ;
- `GET /api/queue/worklists` — listes de travail ;
- `GET /api/resources/board` — état des ressources ;
- `GET /api/resources/eligible` — ressources affectables ;

— `POST /api/care/visits/:id/triage` — triage structuré ;

— `POST /api/care/visits/:id/transfer` — demande de transfert ;

— `GET /api/chief/*` — supervision (pipeline, SLA, alertes) ;

— `GET /api/events/stream` — flux temps réel ;

— `GET /api/ai/recommendations` — recommandations IA.

6.11 Annexe D — Captures d'écran de la plateforme

Les captures d'écran sont à déposer dans le dossier `rapport-stage/images/` avec les noms ci-dessous. Elles apparaissent automatiquement dans les figures du rapport (voir figure 4.1, 4.5, 4.2, 4.3, 4.4 et 4.6).

— `login_screen.png` — page de connexion ;

— `dashboard_receptionniste.png` — dashboard Accueil ;

— `dashboard_infirmier.png` — dashboard Infirmier (triage, plan des lits) ;

— `dashboard_medecin.png` — dashboard Médecin ;

— `dashboard_chef.png` — dashboard Chef de service (supervision) ;

— `dashboard_admin.png` — dashboard Administrateur ;

— `architecture_picu.png` — architecture en trois couches.

6.12 Annexe E — Scénarios et commandes de test

Invariants de ressources (`npm run test:v4`) Vérifie qu'un lit en nettoyage ne peut être affecté, que la libération le bascule en nettoyage et que la transition directe vers « prêt » est refusée.

Règles de supervision (`npm run test:supervision`) Vérifie qu'aucun seuil SLA non approuvé n'est activé.

Intelligence d'enregistrement (`npm run test:registration`) Vérifie les indices d'enregistrement (motif à risque, ambulance, priorité élevée) sans modification de la priorité clinique.

Benchmark de triage `node benchmarks/run-triage-benchmark.js` mesure la latence du modèle NIM sur un jeu de cas et enregistre les résultats.

6.13 Annexe F — Matrice de traçabilité complète

Table 6.3 – Matrice Exigence → Implémentation → Test

Exigence	Implémentation	Validation
Une visite active par patient	`visitService.registerVisitTransaction`	Conflit 409
Un patient par ressource	`assertBedAssignable` + verrou	Test concurrence (T6)
Transitions autorisées	`visitService`, `careStage`	Tests par rôle
Ressource non affectable si nettoyage	`resourceService`	`test:v4` (T1)
Remise en service contrôlée	`setOperationalStatus`	`test:v4` (T3)
Seuils SLA approuvés uniquement	`supervisionRulesService`	`test:supervision` (T4)
Indices d'enregistrement	`registrationHints`	`test:registration` (T5)
Idempotence des actions	`findIdempotentActionResult`	Test idempotence (T7)
Approbation des actions IA	`action-policy`	Tests de politique IA
Traçabilité	`visit_events`	Parcours E2E (T8)